МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена трудового Красного Знамени федеральное государственное бюджетное

образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Отчет по лабораторной работе 7

Многопоточность

Выполнила: студентка группы БПИ2401

Садовникова Евгения

Проверил: Харрасов Камиль Раисович

Москва, 2025

Содержание

Цель работы 3

Задания 3

Ход работы: 4

Задание 1 4

Листинг 1. class TopWords ..... **Ошибка! Закладка не определена.**

Выходные данные: 4

Задание 2 5

Листинг 2. class PasswordVerification**Ошибка! Закладка не определена.**

Листинг 3. class MainStack ..... **Ошибка! Закладка не определена.**

Выходные данные: 5

Задание 3 5

Листинг 4. class SalesStore..... **Ошибка! Закладка не определена.**

Листинг 5. class Product ..... **Ошибка! Закладка не определена.**

Листинг 6. class MainStore..... **Ошибка! Закладка не определена.**

Выходные данные: 7

Контрольные вопросы 9

Вывод..... 9

Ссылка на GitHub: 14

Цель работы:

Изучить коллекции

Задания

Задание 1. Реализация многопоточной программы для вычисления суммы элементов массива.

Вариант 2. Создать пул потоков с помощью класса ExecutorService и разделить массив на равные части, каждую из которых будет обрабатывать отдельный поток. После завершения работы всех потоков результаты будут складываться в главном потоке.

Задание 2. Реализация многопоточной программы для поиска наибольшего элемента в матрице.

Вариант 2. Создать пул потоков с помощью класса ExecutorService и разделить матрицу на равные части, каждую из которых будет обрабатывать отдельный поток. После завершения работы всех потоков результаты будут сравниваться в главном потоке для нахождения наибольшего элемента.

Задание 3. У вас есть склад с товарами, которые нужно перенести на другой склад. У каждого товара есть свой вес. На складе работают 3 грузчика. Грузчики могут переносить товары одновременно, но суммарный вес товаров, переносимый ими за одну итерацию, не может превышать 150 кг. Как только грузчики соберут 150 кг товаров, они отправятся на другой склад и начнут разгружать товары. Напишите программу на Java, используя многопоточность, которая реализует данную ситуацию.

4. Использование Semaphore. Используйте семафоры для ограничения доступа к складу и контроля над весом товаров.

Ход работы:

Задание 1

```
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

/*
InterruptedException
– если главный поток был прерван

ExecutionException
– если внутри Callable.call() возникло исключение
 */
public class Sum {
        public static void main(String[] args) throws
InterruptedException, ExecutionException {
        int[] mas = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
        ExecutorService executor = Executors.newFixedThreadPool(2);
        Future<Integer> firstPart = executor.submit(() -> {
            int sum = 0;
            for (int i = 0; i < mas.length / 2; i++) {
                sum += mas[i];
            }
            return sum;
        });

        Future<Integer> secondPart = executor.submit(() -> {
            int sum = 0;
            for (int i = mas.length / 2; i < mas.length; i++) {
                sum += mas[i];
            }
            return sum;
        });

        int totalSum = firstPart.get() + secondPart.get();
        System.out.println("Общая сумма: " + totalSum);

        executor.shutdown();
    }
}
```

Выходные данные:

Работа программы представлена на рисунке 1.

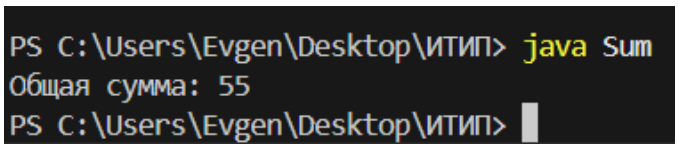

Рисунок 1. Результат программы

Задание 2

```
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class Max {
        public static void main(String[] args) throws
InterruptedException, ExecutionException {
            int[][] mas = { { 3, 7, 8 }, { 7, 1, 3 }, { 5, 6, 9 } };
            ExecutorService executor = Executors.newFixedThreadPool(2);
            Future<Integer> firstPart = executor.submit(() -> {
                int max = mas[0][0];
                for (int i = 0; i < mas.length / 2; i++) {
                    for (int j = 0; j < mas[0].length; j++) {
                        if (max < mas[i][j]) {
                            max = mas[i][j];
                        }
                    }
                }
                return max;
            });

            Future<Integer> secondPart = executor.submit(() -> {
                int max = mas[mas.length / 2][0];
                for (int i = mas.length / 2; i < mas.length; i++) {
                    for (int j = 0; j < mas[0].length; j++) {
                        if (max < mas[i][j]) {
                            max = mas[i][j];
                        }
                    }
                }
                return max;
            });

                System.out.println("Максимальное число: " +
Math.max(firstPart.get(), secondPart.get()));

            executor.shutdown();
        }
}
```

Выходные данные:

Работа программы представлена на рисунке 2.

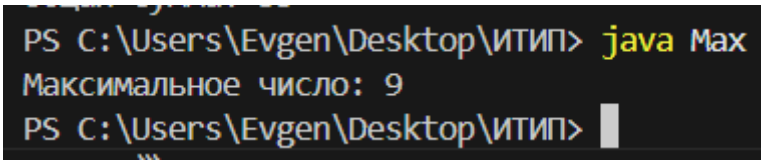

Рисунок 2. Результат программы

Задание 3

```
import java.util.*;
```

```
import java.util.concurrent.Semaphore;

public class Warehouse {

    private static final Queue<Integer> goods = new LinkedList<>();

    private static final Semaphore mutex = new Semaphore(1);
    private static final Semaphore unloadMutex = new Semaphore(1);

    private static int currentWeight = 0;
    private static final int MAX_WEIGHT = 150;

    private static boolean unloading = false;

    public static void main(String[] args) {

        goods.addAll(List.of(
                140, 30, 40, 20, 50, 100, 60, 25, 35, 45, 100));

        new Thread(new Loader(1)).start();
        new Thread(new Loader(2)).start();
        new Thread(new Loader(3)).start();
    }

    static class Loader implements Runnable {

        private final int id;

        Loader(int id) {
            this.id = id;
        }

        @Override
        public void run() {
            try {
                while (true) {

                    mutex.acquire();

                    // Склад пуст → финальная разгрузка
                    if (goods.isEmpty()) {
                        mutex.release();
                        unload();
                        break;
                    }

                    int weight = goods.peek();

                   // Если товар не помещается — сначала разгружаемся
                    if (currentWeight + weight > MAX_WEIGHT) {
                        mutex.release();
                        unload();
                        continue;
                    }

                    // Берём товар
```

```
                        goods.poll();
                        currentWeight += weight;

                        System.out.println(
                                "Грузчик " + id +
                                        " взял товар весом " + weight +
                                        " кг (всего " + currentWeight +
" кг)");

                       boolean needUnload = currentWeight == MAX_WEIGHT;
                        mutex.release();

                        if (needUnload) {
                            unload();
                        }

                        Thread.sleep(300);
                    }

                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }

            private void unload() throws InterruptedException {

                unloadMutex.acquire();

                if (unloading || currentWeight == 0) {
                    unloadMutex.release();
                    return;
                }

                unloading = true;
                int weightToUnload = currentWeight;

                System.out.println(
                        "\nГрузчики везут " + weightToUnload + " кг на
другой склад...");

                currentWeight = 0;
                unloading = false;

                System.out.println(
                        "Разгрузка завершена. Грузчики вернулись.\n");

                unloadMutex.release();
                Thread.sleep(300);
            }
        }
    }
```

Выходные данные:

Работа программы представлена на рисунке 3.

```
PS C:\Users\Evgen\Desktop\ИТИП> java Warehouse
Грузчик 1 взял товар весом 140 кг (всего 140 кг)

Грузчики везут 140 кг на другой склад...
Разгрузка завершена. Грузчики вернулись.

Грузчик 3 взял товар весом 30 кг (всего 30 кг)
Грузчик 3 взял товар весом 40 кг (всего 70 кг)
Грузчик 1 взял товар весом 20 кг (всего 90 кг)
Грузчик 2 взял товар весом 50 кг (всего 140 кг)

Грузчики везут 140 кг на другой склад...
Разгрузка завершена. Грузчики вернулись.

Грузчик 1 взял товар весом 100 кг (всего 100 кг)

Грузчики везут 100 кг на другой склад...
Разгрузка завершена. Грузчики вернулись.

Грузчик 3 взял товар весом 60 кг (всего 60 кг)
Грузчик 2 взял товар весом 25 кг (всего 85 кг)
Грузчик 1 взял товар весом 35 кг (всего 120 кг)

Грузчики везут 120 кг на другой склад...
Разгрузка завершена. Грузчики вернулись.

Грузчик 3 взял товар весом 45 кг (всего 45 кг)

Грузчики везут 45 кг на другой склад...
Разгрузка завершена. Грузчики вернулись.

Грузчик 3 взял товар весом 100 кг (всего 100 кг)

Грузчики везут 100 кг на другой склад...
Разгрузка завершена. Грузчики вернулись.
```

Рисунок 3. Результат программы

Контрольные вопросы

1. Как реализуется многопоточность в Java?

Многопоточность в Java реализуется через:

- Класс Thread (наследование)
- Интерфейс Runnable (реализация)
- Интерфейс Callable (с возвращаемым значением)
- Использование ExecutorService и пулов потоков
- Использование библиотеки Executors и лямбда-выражений

2. Что такое поток?

Поток (thread) — это легковесный процесс, наименьшая единица выполнения в Java. Потоки разделяют общее адресное пространство процесса, но имеют собственные стеки вызовов и регистры. JVM позволяет создавать и управлять множеством потоков параллельно.

3. Для чего нужно ключевое слово synchronized?

synchronized используется для:

- Обеспечения взаимного исключения (mutual exclusion) доступа к общим ресурсам
- Гарантии атомарности операций
- Синхронизации потоков при доступе к критическим секциям
- Предотвращения состояния гонки (race condition)

```
public synchronized void method() { ... }
```

4. Для чего нужно ключевое слово volatile?

volatile обеспечивает:

- Видимость изменений переменной между потоками (чтение из основной памяти)
- Запрет на кэширование переменной в регистрах процессора
- Упорядочивание операций (happens-before)

Но не гарантирует атомарность сложных операций.

5. Зачем нужно синхронизировать потоки?

Синхронизация нужна для:

- Предотвращения race condition при доступе к общим данным
- Обеспечения консистентности данных
- Реализации взаимодействия между потоками (producer-consumer)
- Избежания deadlock и livelock ситуаций

6. Какие есть способы синхронизации потоков?

- Мониторы объектов (synchronized методы и блоки)
- Атомарные классы (AtomicInteger, AtomicReference, etc.)
- Lock интерфейсы (ReentrantLock, ReadWriteLock)
- Синхронизаторы (CountDownLatch, CyclicBarrier, Semaphore, Exchanger)
- Потокобезопасные коллекции (ConcurrentHashMap, CopyOnWriteArrayList)
- Ключевое слово volatile (ограниченная синхронизация)

7. В чем разница между Thread и Runnable?

Thread Runnable

КлассИнтерфейс

Наследование (extends) Реализация (implements)

Одно наследование в Java Можно реализовать несколько интерфейсов

Менее гибкий Более гибкий и рекомендуемый подход

start() создает новый поток Требует передачи в конструктор Thread

Для Runnable нет ограничения единственного наследования, есть возможность использования пулов потоков.

8. Какие состояния может иметь поток? Опишите жизненный цикл потока.

Состояния потока:

- NEW — создан, но не запущен (new Thread())
- RUNNABLE — готов к выполнению или выполняется (start())
- BLOCKED — заблокирован, ждет монитор

- WAITING — ждет сигнала без таймаута (wait(), join())
- TIMED_WAITING — ждет с таймаутом (sleep(), wait(timeout))
- TERMINATED — завершил выполнение

Жизненный цикл: создание, выполнение, ожидание, блокировка и завершение.

9. Что такое daemon-поток? Как его создать?

Daemon-поток — фоновый поток, который автоматически завершается, когда все не-daemon потоки завершились. Не мешает завершению JVM.

Создание:

```
Thread daemonThread = new Thread(() -> { ... });
daemonThread.setDaemon(true); // ДО start()!
daemonThread.start();
```

10. Как принудительно остановить поток?

Устаревшие методы (deprecated):

- stop() — опасен, может оставить данные в некорректном состоянии
- suspend() и resume() — могут вызвать deadlock

Современные пордходы:

- Флаг завершения:

```
volatile boolean running = true;

public void run() {
   while(running) { ... }
}

public void stop() {
   running = false;
}
```

- Прерывание:

```
thread.interrupt();
```

```
public void run() {
   while(!Thread.currentThread().isInterrupted()) { ... }
}
```

- Future.cancel() при использовании ExecutorService

11. Как работает метод join()? Для чего он используется?

join() заставляет текущий поток ждать завершения другого потока:

```
Thread t = new Thread(() -> { ... });
t.start();
t.join(); // текущий поток ждет завершения t
```

Используется для:

- Ожидания завершения параллельных вычислений
- Синхронизации порядка выполнения потоков
- Сбора результатов из дочерних потоков

12. Что такое «гонка данных» (race condition)?

Race condition — ошибка проектирования многопоточной системы, когда результат выполнения зависит от порядка выполнения потоков:

```
// Два потока инкрементируют одну переменную
if (count == 10) {  // Поток 1: count = 10
   count++;        // Поток 2 тоже считает count = 10
}                  // Результат: 12 вместо 11!
```

13. Что такое deadlock? Как его избежать?

Deadlock — ситуация, когда два или более потока бесконечно ждут друг друга, заблокировав взаимно нужные ресурсы.

Условия deadlock (Коффмана):

- Взаимное исключение

- Удержание и ожидание
- Отсутствие вытеснения
- Циклическое ожидание

Как избежать:

- Упорядочивание блокировок (всегда в одном порядке)
- Использование таймаутов (tryLock(timeout))
- Избегание вложенных блокировок
- Использование Thread.join() вместо взаимных блокировок

14. Что такое wait(), notify() и notifyAll()? В каком классе они объявлены?

Эти методы объявлены в классе Object и используются для взаимодействия потоков:

- wait() — освобождает монитор и переводит поток в WAITING
- notify() — будит один случайный поток, ждущий на этом мониторе
- notifyAll() — будит все потоки, ждущие на этом мониторе

Все эти методы должны вызываться внутри synchronized блока.

```
synchronized(lock) {
   while(!condition) {
      lock.wait();  // освобождает lock и ждет
   }
   // работа
   lock.notifyAll(); // будит ждущие потоки
}
```

15. Что такое ThreadPool? Какие реализации ExecutorService есть в Java?

ThreadPool — пул переиспользуемых потоков для выполнения задач. Эффективнее создания новых потоков для каждой задачи:

Реализации ExecutorService:

- Executors.newFixedThreadPool(n) — фиксированное количество потоков
- Executors.newCachedThreadPool() — создает потоки по мере необходимости, переиспользует
- Executors.newSingleThreadExecutor() — один поток (очередь задач)
- Executors.newScheduledThreadPool(n) — пул для планирования задач
- Executors.newWorkStealingPool() — work-stealing алгоритм (параллелизм)

```
ExecutorService executor = Executors.newFixedThreadPool(4);
executor.submit(() -> { /* задача */ });
executor.shutdown();
```

Преимущества:

- Снижение накладных расходов на создание потоков
- Контроль над количеством параллельных задач
- Очередь задач для обработки
- Управление жизненным циклом потоков

Вывод

В ходе выполнения лабораторной работы изучена многопоточность

Ссылка на GitHub: https://github.com/Evgeshhha/Information-technology-and-programming